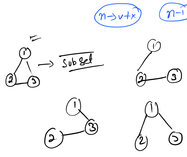
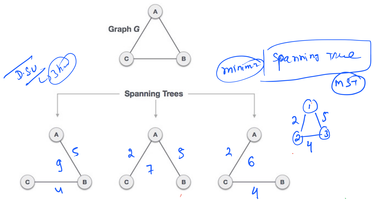
Lec-33
16/09/25 9:20 AM

Spanning Tree



Spanning Tree

A spanning tree is a subset of Graph G, which has all the vertices covered with minimum possible number of edges. Hence, a spanning tree does not have cycles and it cannot be disconnected..



Properties of Spanning Tree

- A connected graph G can have more than one spanning tree.
- Spanning Tree has n-1 edges, where n is the number of Node (Vertices)
- All possible spanning trees of graph G, have the same number of edges and vertices.
- The spanning tree does not have any cycle (loops).
- Removing one edge from the spanning tree will make the graph disconnected, i.e. the spanning tree is minimally connected.
- Adding one edge to the spanning tree will create a circuit or loop, i.e. the spanning tree is maximally acyclic.

Minimum Spanning Tree (MST)

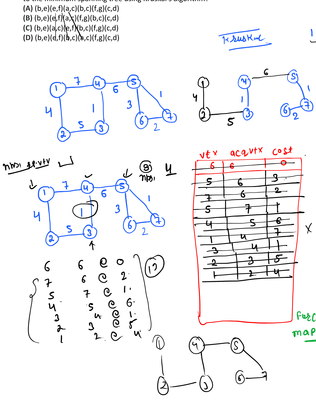
In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.

- Kruskal's Algorithm

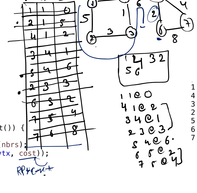
Prim's Algorithm
Both are greedy algorithms.

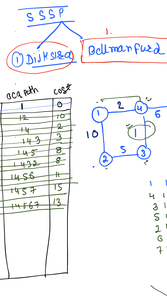
Consider the following graph:

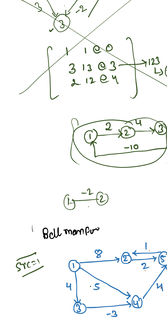
Which one of the following is NOT the sequence of edges added to the minimum spanning tree using Kruskal's algorithm?
(A) (b,e)(e,f)(a,c)(b,c)(f,g)(c,d)
(B) (b,e)(e,f)(a,c)(f,g)(b,c)(c,d)
(C) (b,e)(a,c)(e,f)(b,c)(f,g)(c,d)
(D) (b,e)(e,f)(b,c)(a,c)(f,g)(c,d)

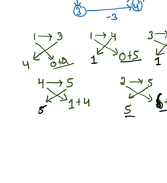


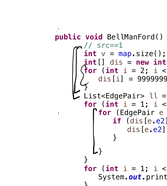
```
public int PrimsAlgorithm() {
    PriorityQueue<PrimsPair> pq = new PriorityQueue<>((a,b)->a.cost-b.cost);
    HashSet<Integer> visited = new HashSet<>();
    pq.add(new PrimsPair(1, 1, 0));
    int sum = 0;
    while (!pq.isEmpty()) {
        // 1. remove
        PrimsPair rp = pq.poll();
        // 2. Ignore if Already visited
        if (visited.contains(rp.vtx)) {
            continue;
        }
        // 3. Marked visited
        visited.add(rp.vtx);
        sum += rp.cost;
        // 4. self work
        System.out.println(rp);
        // 5. add unvisited nbrs
        for (int nbrs : map.get(rp.vtx).keySet()) {
            if (!visited.contains(nbrs)) {
                int cost = map.get(rp.vtx).get(nbrs);
                pq.add(new PrimsPair(nbrs, rp.vtx, cost));
            }
        }

    }
    return sum;
}
```

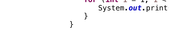












```
public void BellManFord() {
    // src=1
    int v = map.size();
    int[] dis = new int[v + 1];
    for (int i = 2; i < dis.length; i++) {
        dis[i] = 9999999;
    }
    List<EdgePair> ll = getalledge();
    for (int i = 1; i < v; i++) {
        for (EdgePair e : ll) {
            if (dis[e.e2] > dis[e.e1] + e.cost) {
                dis[e.e2] = dis[e.e1] + e.cost;
            }
        }
    }
    for (int i = 1; i < dis.length; i++) {
        System.out.print(dis[i] + " ");
    }
}
```